面向技能库规模控制的在线聚类与层次化 Skill 自进化框架

An Online Clustering and Hierarchical Self-Evolution Framework for Scalable Agent Skill Libraries

> **文档性质：** 研究方案与方法设计稿（非已完成实验的论文）
> **版本：** v1.1 · 2026-10-11（修订研究动机与总体流程）
> **范围约定：** 本版本聚焦 Skill 的生成、聚类、整合、晋升、拆分及索引维护；**暂不设计自动化行为效果验证模块。**

摘要

随着大语言模型智能体在持续任务交互中积累可复用经验，自动生成 Skill 为长期能力演化提供了可行路径。然而，若将每条新经验直接保存为独立 Skill，技能库容易出现数量持续增长、内容重复、检索歧义与维护负担增加等问题。为缓解这一问题，本文首先考虑对候选 Skill 进行在线语义聚类，以识别具有共同任务目标的能力群。但聚类只能回答"哪些技能属于同一类"，不能直接解决"簇内技能如何组织和调用"。因此，本文进一步引入**父子 Skill 组织思想**：将同簇技能中的共享能力抽象为可独立发现的父 Skill，将差异化的专项流程组织为按需加载的子能力。

在此基础上，本文分析层次化组织面临的两类次生问题：一方面，若持续将子能力的完整内容追加到父 Skill，父 Skill 将逐渐冗长；另一方面，即使通过外部文件承载专项内容，子能力的持续积累仍可能造成内部模块膨胀，并固化不合理的任务边界。为此，本文采用 **Master Skill + References** 的两级文件结构，其中 Master Skill 维护通用流程与子能力索引，References 承载详细操作策略；进一步设计基于任务独立性与簇内语义分化的**动态晋升（Promotion）与拆分（Splitting）**机制，允许成熟子能力形成独立 Skill，并按需重组过宽的能力簇。最后，通过局部更新 Skill Embedding、簇中心与检索索引，保持结构变化与技能发现的一致性。本文给出框架设计、增量维护流程和潜在研究问题；当前为待实现方案，尚无实验性能结论，也暂不包含自动化行为效果验证。

关键词： Agent Skills；自进化智能体；在线聚类；父子技能；渐进式加载；动态晋升；技能库管理

1. 引言

1.1 背景

Agent Skill 通常以 `SKILL.md` 为入口，利用 `name`、`description` 声明任务适用范围，并通过正文及附属资源封装可复用的操作流程。相比将所有操作策略固定在系统提示词中，Skill 机制有利于将经验组织为可按需调用的能力单元 [1]。

当智能体具备从执行轨迹中总结经验、自动创建和修订 Skill 的能力时，Skill 库不再是静态配置，而成为持续增长的知识集合。然而，这种增长并不天然带来有效的能力扩展：同一类任务可能产生多个描述不同、操作内容却高度重合的 Skill，导致技能入口变多、维护关系复杂化。

1.2 第一重动机：抑制 Skill 库数量膨胀

若采用"**新经验 → 新 Skill**"的直接积累策略，技能数量可能随任务次数持续增长。其主要问题包括：

1. **能力冗余：** 多个 Skill 解决高度相似的问题，重复保存工作流程。
2. **技能选择歧义：** 类似的 `description` 使候选 Skill 难以区分，增加误选风险。

3. **维护成本提高：** 相似策略被分别修订，可能产生不一致的版本。
4. **发现与上下文开销：** 即使采用元数据级发现，活跃技能数量增长仍会增加检索和管理负担。

因此，本文首先考虑**在线 Skill 聚类**：对于持续生成的候选 Skill，先依据其任务描述判断是否可归入已有能力簇；仅当找不到合适的能力簇时，才建立新的独立 Skill 入口。该策略旨在把"每次生成一个独立入口"转变为"优先复用已有能力类别"。

然而，**聚类本身只建立相似关系，并不规定簇内多个 Skill 如何协作、如何存储或如何对外提供统一的任务入口。**要把聚类结果转化为真正可用的技能库，还需要进一步确定簇内技能的组织形式。

1.3 从在线聚类到父子 Skill 组织

对于语义相近的多个 Skill，它们往往既包含相同的任务目标与通用步骤，也包含面向不同场景的专项操作。例如，"数据清洗""异常值处理"和"缺失值修复"可能共享数据预处理的总体目标，但各自具有不同的操作条件与细节。若将三者作为平级独立 Skill，容易产生重复的触发入口；若简单拼接成一个长文档，又会混淆共享流程与专项规则。

因此，本文引入**父子 Skill 的层次化组织思想**：

- **父 Skill（Parent Skill）：** 归纳同一能力簇的共同任务范围、通用流程与约束，作为面向外部任务的统一入口。
- **子 Skill（Child Skill）：** 保留簇内某项专项能力的适用条件、操作步骤和边界规则，供父 Skill 在需要时选择性调用。

这里的"子 Skill"表示**逻辑上的能力模块**，不要求它在初始阶段就具有独立的 `SKILL.md` 或注册入口。初始的独立 Skill 在聚类整合后，可以被吸收为某个子能力，也可以为新簇建立父 Skill；并非所有被判定相似的 Skill 都必须合并，职责独立者仍可单独成簇。

由此得到第一层组织关系：

> **在线聚类负责确定"属于哪类能力"，父子 Skill 组织负责确定"共同流程由谁承担、专项能力由谁保存"。**

为避免术语混淆，下文将每个活跃能力簇中**可独立发现的父 Skill**称为 **Master Skill**，将其管理的专项子能力称为 **Reference Module**（以 `references/*.md` 实现）。这是一种基于标准 Skill 目录的工程组织约定，而不是 Agent Skills 规范中新增的原生 Skill 类型。

1.4 第二重动机：防止父 Skill 内容膨胀

建立父子关系后，还需要回答一个实际问题：**子 Skill 的执行细节究竟放在哪里？** 如果在线归类后不断把所有新技能的完整指令追加到父 `SKILL.md`，虽然活跃入口数量下降，父 Skill 却会逐渐冗长，形成由"技能数量膨胀"向"单个技能内容膨胀"的转移。具体表现为：

- 通用流程与大量场景特例混杂，使父 Skill 难以保持清晰的能力边界；
- 每次调用父 Skill 都可能加载当前任务并不需要的专项规则；
- 对局部经验的修改牵动主文件结构，增加维护复杂性；
- 父 Skill 的 `description` 可能因子能力积累而持续扩张，影响任务选择。

为此，本文采用 **Master Skill + References** 的两级文件结构：Master `SKILL.md` 只保留稳定的共享流程、必要约束与子能力索引，专项规则则存放在 `references/`，执行时按需读取。该设计利用 Agent Skills 的渐进式披露机制 [1]，使父子关系能够在文件层面落地，而不必把所有子能力合并成一份长文档。

1.5 第三重动机：防止子能力膨胀与任务边界固化

将专项能力下沉至 References 并不意味着规模问题已经消失。随着任务经验不断积累，同一个 Master 下可能出现越来越多的 Reference；同时，某些最初只是内部步骤的子能力，可能逐渐形成可被用户直接请求的完整工作流。如果仍将它们固定为从属模块，父 Skill 就会变成覆盖多个独立任务的调度入口，而独立能力也无法被直接发现。

因此，本文引入**动态晋升（Promotion）与簇拆分（Splitting）**，使父子关系能够随能力演化而调整：

- **晋升：** 当某个 Reference 具备独立任务意图和完整执行流程时，将其转化为可独立触发的 Skill。
- **拆分：** 当一个能力簇内部形成多个职责相对独立的子群时，重新划分簇并配置相应的 Master Skill。
- **合并与归档：** 对仍然重复的 References 或簇进行整合，避免结构重组反过来造成新的入口冗余。

这样，层次关系并非静态的"父管理子"，而是允许子能力**吸收、分化、晋升和重组**的动态组织结构。

综上，本文的研究思路遵循一条由问题驱动的递进链条：

> **独立 Skill 数量膨胀 → 在线聚类 → 需要组织簇内能力 → 父子 Skill 结构 → 父 Skill 内容膨胀 → Master + References 渐进式加载 → 子能力数量及职责边界失衡 → 动态晋升、拆分与合并。**

1.6 拟研究的主要贡献

1. 提出面向候选 Skill 的**在线语义聚类与增量归类机制**，减少语义重复的独立技能入口。
2. 设计从能力簇到**父子 Skill 层次组织**的映射方式，并通过 Master Skill 与 References 分离共享流程和专项步骤。
3. 提出由**任务独立性与簇内语义分化**驱动的动态晋升、拆分及合并机制，修正长期积累过程中不合理的能力边界。
4. 建立结构变更后的**局部 Embedding 与聚类索引同步机制**，保持技能内容、成员归属和检索结果的一致性。

上述内容属于**方法设计与预期研究点**，不是已经获得实验证实的贡献。



2. 问题定义与总体框架

2.1 基本定义

给定持续到达的任务序列 $\mathcal{T} = \{t_1, t_2, \ldots\}$，系统仅在出现值得沉淀的任务经验时生成候选技能 s_i。一个候选 Skill 至少包含 `name`、`description` 与操作正文；候选是**尚未确定归属的能力单元**，并非自动发布的独立技能入口。

定义活跃技能池为：

$$\mathcal{P} = \{C_1, C_2, \ldots, C_K\},$$

其中每个能力簇表示为：

$$C_k = (M_k, \mathcal{R}_k, \mu_k, \mathcal{H}_k).$$

- M_k：能力簇的**父 Skill / Master Skill**，是可独立发现的 `SKILL.md` 入口；
- $\mathcal{R}_k$：隶属于父 Skill 的**子能力集合**，以 `references/*.md` 承载，不单独注册为 Skill；
- μ_k：根据历史接纳的 Skill 表示维护的语义簇中心，用于新候选归类；
- $\mathcal{H}_k$：被接纳的历史能力表示、来源、归属与变更记录，供中心更新和结构重组使用。

这一区分保证了**聚类关系**与**文件层次关系**不被混为一谈：簇中心决定候选更接近哪个能力群；Master 为外部任务提供入口；References 保存内部专项步骤。新簇最初可以只有 Master、没有 Reference，待同类能力逐步积累后再形成父子结构。

2.2 优化目标

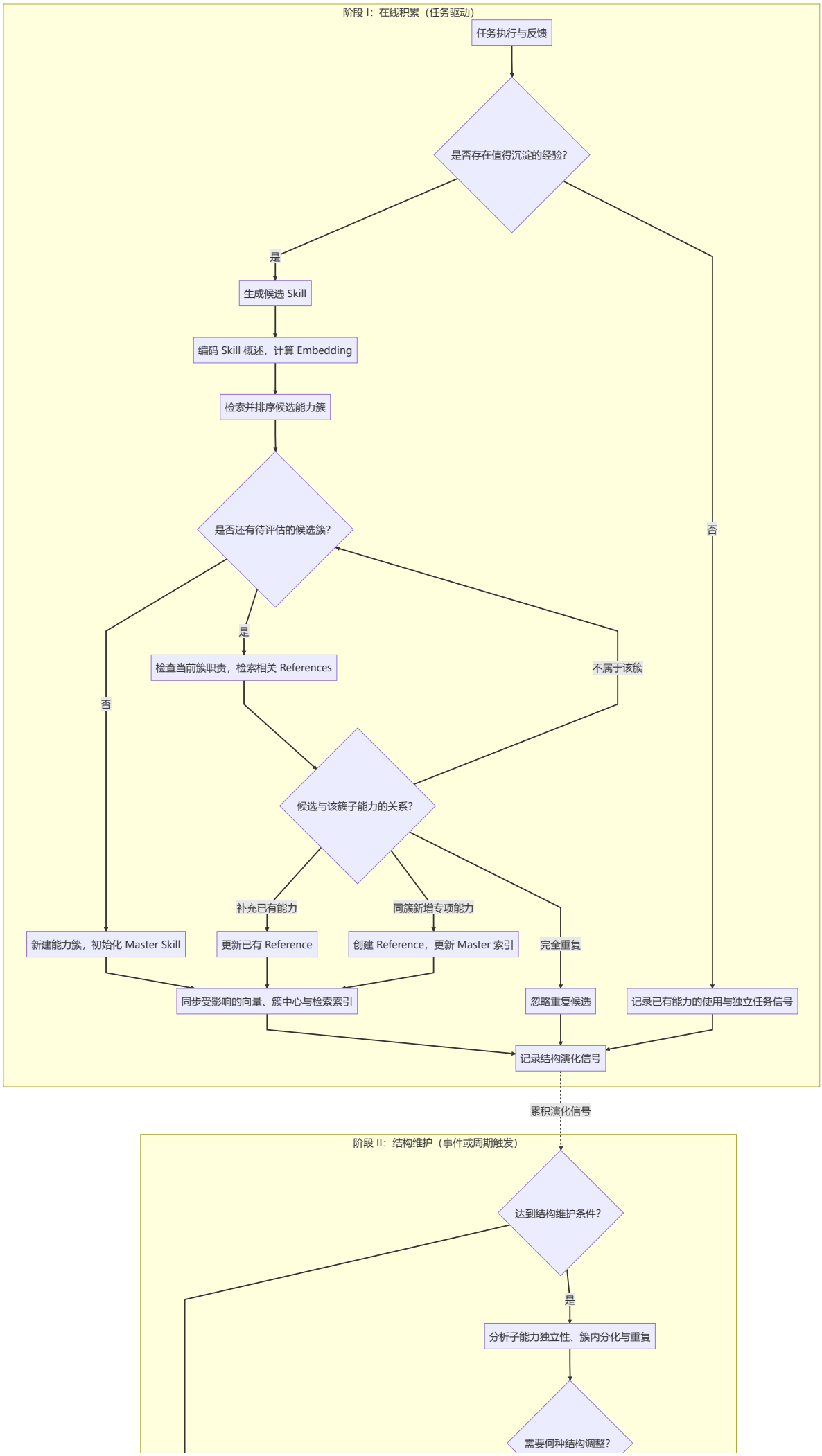
框架希望在保留任务能力边界的同时，降低活跃 Skill 数量、内容冗余以及执行时的上下文开销。可将其表示为一个概念性多目标问题：

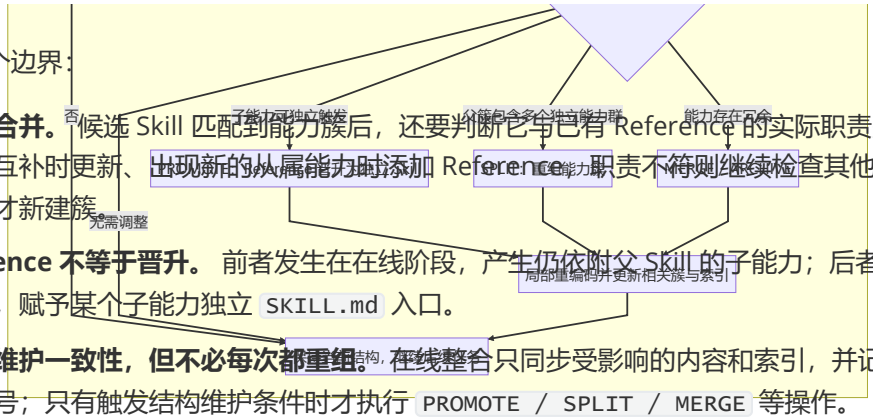
$$\min \ \alpha N_{\text{active}} + \beta D_{\text{redundancy}} + \gamma B_{\text{context}} + \delta A_{\text{ambiguity}},$$

其中分别表示活跃技能数、内容冗余、上下文预算消耗与技能路由歧义。该式用于说明设计目标，**目前不作为已经实现的训练损失或具有确定权重的优化函数**；同时，任何压缩都不应以破坏必要的任务边界为代价。

2.3 总体流程

框架由**在线技能积累**与**低频结构维护**两条相互衔接、但执行频率不同的流程组成。在线积累首先决定是否有必要生成候选 Skill，再进行簇匹配与父子能力整合；结构维护不随每次候选整合同步执行，而是依据累计的任务独立性、能力分化或周期性事件，决定是否需要调整既有层次结构。


阶段 I：在线积累（任务驱动）
任务执行与反馈
是否存在值得沉淀的经验?
是
否
生成候选 Skill
编码 Skill 概述，计算 Embedding
检索并排序候选能力簇
是否还有待评估的候选簇?
是
否
检查当前簇职责，检索相关 References
不属于该簇
候选与该簇子能力的关系?
补充已有能力
同簇新增专项能力
完全重复
新建能力簇，初始化 Master Skill
更新已有 Reference
创建 Reference，更新 Master 索引
同步受影响的向量、簇中心与检索索引
忽略重复候选
记录已有能力的使用与独立任务信号
记录结构演化信号
累积演化信号
阶段 II：结构维护（事件或周期触发）
达到结构维护条件?
是
分析子能力独立性、簇内分化与重复
需要何种结构调整?



图中需要强调三个边界：

1. **聚类不等于合并。** 候选 Skill 匹配到能力簇后，还要判断它与已有 Reference 的实际职责关系；重复时忽略、互补时更新、出现新的从属能力时添加 Reference，职责不符则继续检查其他候选簇，均不适合时才新建簇。
2. **创建 Reference 不等于晋升。** 前者发生在在线阶段，产生仍依附父 Skill 的子能力；后者发生在结构维护阶段，赋予某个子能力独立 `SKILL.md` 入口。
3. **每次变化都维护一致性，但不必每次都重组。** 在线整合只同步受影响的内容和索引，并记录后续判断所需的信号；只有触发结构维护条件时才执行 `PROMOTE / SPLIT / MERGE` 等操作。

图中从"记录结构演化信号"到"是否达到维护条件"的虚线仅表示**数据依赖与触发判断**，不代表每次在线更新后都立刻运行全套重组。实际实现可在后续任务入口或批量维护命令中检查条件，满足条件时才执行结构调整。

---

3. 基于任务反馈的候选 Skill 生成

为减少无效技能增长，系统不应在每次任务结束后都自动生成新的 Skill，而应先检测经验沉淀信号，包括：

- 用户明确要求保存某种可复用方法；
- 某次执行暴露并修复了可迁移的错误处理方式；
- 同类任务反复出现稳定的可复用操作模式；
- 某个专项流程首次形成相对清晰的适用条件与操作步骤。

满足条件后，Skill Generator 从任务意图、执行轨迹、有效步骤与关键约束中生成候选 `SKILL.md`。候选 Skill 暂不作为独立活跃入口发布，而是进入在线聚类流程。

本方案的聚类对象明确为**候选 Skill**，不是每次任务的 Experience Case。Raven 的经验驱动 Skill 演化提供了从任务经验中提炼可复用程序的参考 [2]，但本方案在**已生成候选 Skill 之后**进行语义聚类，二者的数据组织粒度不同。

---

4. 在线 Skill 聚类

4.1 基于概述的 Skill 表示

使用 `SKILL.md` 的标准元数据 `name` 和 `description` 构建语义表示，避免直接编码冗长正文及通用指令。基础版本定义：

$$z_i = \mathrm{Norm}(\mathrm{Encoder}(\mathrm{name}(s_i) \oplus \mathrm{description}(s_i))),$$

其中 $\oplus$ 表示文本拼接，`Encoder` 可采用 BGE-M3 等文本嵌入模型。后续也可将触发条件与核心能力摘要纳入编码文本，但应保持全池一致的表示协议。

4.2 在线匹配与新簇创建

对于候选技能向量 z_i，计算其与各簇中心的余弦相似度：

$$k^* = \arg\max_k \cos(z_i, \mu_k).$$

采用**相似度召回 + 内容关系判断**的两阶段策略：

1. 按相似度检索并排序若干候选簇，优先考虑达到可配置阈值 τ 的簇；边界情况可由 LLM 判断职责范围；
2. 对当前候选簇检查任务边界，并匹配其内部 Reference；语义相似但职责不符时，继续考察下一个候选簇；
3. 只有在候选簇均不适合时，才创建新的能力簇与 Master Skill；
4. **语义相似仅用于候选簇召回，不能直接推导出子能力之间可以融合。**

这里借鉴的是 DP-Means 式"距离不足够近则创建新簇"的**在线增量思想**，不声称该规则本身等价于严格执行某一标准 DP-Means 算法。

4.3 簇中心的增量维护

一个簇由被正式吸收且具有独立信息增量的历史 Skill 表示构成。令其历史向量为 $z_1, \ldots, z_n$，则余弦检索使用的中心为：

$$\mu_k = \mathrm{Norm}\left(\frac{1}{n}\sum_{j=1}^{n} z_j\right).$$

当新能力表示 z_{n+1} 被接纳时，可维护未归一化的累积和 u_k 与成员数 n：

$$u'_k = u_k + z_{n+1}, \qquad \mu'_k = \frac{u'_k}{\|u'_k\|_2}.$$

实现中应区分"**重复候选**"与"**新增有效信息**"：前者直接忽略，不应因为大量重复表述而反复增加簇中心权重。保存历史成员向量使后续拆分、迁移与重算成为可能。

---

5. Master Skill 与 References 的层次化整合

5.1 文件组织

```
skills/
└── master-skill/
    ├── SKILL.md                # 通用流程、约束、触发范围、子能力索引
    ├── references/
    |   ├── capability-a.md     # 专项流程 A
    |   ├── capability-b.md     # 专项流程 B
    |   └── capability-c.md     # 专项流程 C
    ├── scripts/                # 可选：确定性执行逻辑
    └── assets/                 # 可选：资源、模板等
```

第 1.3 节引入的父子 Skill 关系在文件层面落实为 **Master Skill + References**：每个活跃能力簇对应**一个可独立发现的父 Skill**，其主文件 `SKILL.md` 只描述共享流程、必要约束以及子能力的加载条件；不同子能力的专项规则、案例处理和操作步骤则由 References 承载。

Reference 在本方案中被视为**内部子技能模块**：它可以具有适用场景、前置条件、执行步骤与输出约束，但不具备独立注册与触发入口。**子 Skill 是语义上的能力单元，Reference 是其文件实现**；只有执行晋升后，子能力才会成为具有独立元数据与入口的正式 Skill。

5.2 簇内整合决策

新候选归入某簇后，首先与该 Master 下已有 Reference 进行语义匹配，再根据内容关系采取不同操作：

情况	操作	结果
已有内容可完整覆盖	`IGNORE`	不新增文件，不增加活跃能力入口
为某专项流程补充规则或边界情况	`UPDATE_REFERENCE`	局部修订已有 Reference
同属父技能职责，具有新的从属专项流程	`ADD_REFERENCE`	创建内部子能力文件，并更新主索引（不直接晋升）
与当前簇职责不一致	`NEW_CLUSTER`	创建新的候选 Master Skill

通过“**优先更新，必要时新增**”的原则，减少参考文件冗余。Master 的正文不重复复制 Reference 的细节，避免同一策略出现多个事实来源。

5.3 执行时的渐进式加载

采用三级读取路径：

`Skill name/description 发现 → 加载 Master SKILL.md → 根据当前任务按需读取相关 Reference`

这里应注意：**标准 Agent Skills 并不保证自动根据子技能索引完成路由**，实际系统需要提供显式的资源读取接口或由 Agent 按主文件指引调用。子能力结构属于本文设计的管理策略，而非对标准格式的更改。

---

6. 结构自适应演化：动态晋升与拆分

6.1 为什么需要结构重组

当某个 Reference 被持续补充后，它可能从“父 Skill 的内部子能力”演化为“能够独立解决一类任务的完整能力”。与此同时，某些 Master Skill 可能逐渐覆盖多个互不依赖的任务群。这意味着仅靠 Reference 更新仍会产生**内部模块膨胀、触发入口受限和主技能职责泛化**问题。

因此，系统不应将聚类结果永久固定，而应允许技能能力边界随使用情况动态调整。

6.2 晋升（Promotion）

定义： 将一个具有独立任务意图与相对完整执行流程的 Reference，提升为具有自身 `name`、`description` 和 `SKILL.md` 的独立技能入口。

建议触发信号：

1. **任务独立性：** 存在直接以该 Reference 能力为目标的用户请求，而不仅是父 Master 工作流中的一个步骤；
2. **流程完整性：** 已具备清晰的输入、操作步骤、输出与适用边界；
3. **持续复用：** 在不同任务中反复发挥独立作用，或多次独立吸收新增能力。

三个条件中，以**任务独立性**为首要判断依据；内容长度或更新次数只作为提醒信号，不能单独决定晋升。

晋升步骤：

1. 将 Reference 标记为 `PROMOTION_CANDIDATE`；
2. 由 LLM 判断它是否具备独立任务边界，输出 `KEEP / REFACTOR / PROMOTE`；
3. 选择 `PROMOTE` 时，从原 Reference 提取可独立执行的任务范围与流程，构建候选 `SKILL.md` 及其 `name`、`description`；
4. 将晋升候选与其他现有能力簇进行匹配：若已有 Master 能完整承接此能力，则转为跨簇整合，不重复创建独立入口；否则新建独立 Skill 簇；
5. 从原 Master 的内部索引中移除原子能力的重复完整流程，若原父流程仍需它，保留适当的协作或调用说明；
6. 同步受影响 Skill 的 Embedding、簇成员关系与检索索引，并记录来源关系。

重要区别： 晋升不等于单纯移动文件；它赋予原 Reference **独立发现、直接触发和独立维护的能力边界**。

6.3 拆分（Splitting）

定义： 当一个 Master Skill 所在簇包含多个语义上相对分离、执行上能够独立成立的能力群时，将其重组为两个或多个新的技能簇。

建议触发信号：

- 簇内 Reference 或历史候选向量逐渐呈现多个相互分离的群组；
- 主文件为了覆盖所有专项任务，不断扩张触发描述和调度规则；
- 部分子能力已形成各自独立的任务入口、工作流与复用模式；
- 新 Skill 长期以较低相似度勉强归入同一簇。

拆分时先识别相对独立的能力群，再分别构建 Master Skill、迁移相关 References 和历史归属，并重算受影响簇的中心。**不能只依据低余弦相似度自动拆分**：同一工作流内合理存在的互补步骤并不一定应成为不同的 Skill。

6.4 晋升与拆分的区别

比较维度	PROMOTE	SPLIT
操作对象	某一个具备独立性的 Reference	整个职责过宽的能力簇
核心问题	内部能力不能直接触发	一个 Master 同时管理多个独立任务群
主要结果	产生一个独立 Skill，调整父子关系	重新划分多个簇及其 Master/References
优先信号	直接任务需求、流程完整性	簇内语义分化、职责边界不清

6.5 动态触发策略

采用"**在线记录信号、低频执行重组**"的原则，以降低频繁调用 LLM 进行全局重构的成本。

- **在线阶段：** 每次候选整合后仅更新被影响簇的统计信息，包括 Reference 更新次数、独立任务需求、语义离散程度等。
- **事件触发：** 当某个 Reference 多次被独立请求，或簇内持续出现独立能力群时，将其加入重组候选队列。

- **周期触发：** 每累计一定数量的新增/更新事件，检查受影响簇是否需要 `MERGE / PROMOTE / SPLIT / ARCHIVE`。
- **冷却期：** 新晋升或拆分的 Skill 在短期内不参与反向自动合并，避免结构在两个状态间反复震荡。

作为**可调整的初始启发式**，可先考虑"3 次有代表性的独立任务需求触发晋升审查"及"每累计 10 次正式结构变更触发局部维护"，这些数值不是理论最优值，也不是晋升的充分条件。

6.6 其他结构操作

- `MERGE`：合并重复或边界高度重叠的 References、能力簇；
- `REFACTOR`：压缩同一 Reference 内的冗余表达，但保留其归属；
- `ARCHIVE`：停止激活被替代的入口，保留来源和版本记录；
- `KEEP`：保持结构，避免因短期信号过度重组。

---

7. Embedding 与索引的一致性维护

技能内容变化与结构变化必须同步反映到检索层，否则可能出现：某个 Reference 已晋升为新 Skill，但路由索引仍将相关请求指向旧 Master。

结构操作	所需索引变更
`UPDATE_REFERENCE`	若维护 Reference 级索引，则重编码该 Reference 的概述；必要时更新 Master 描述
`ADD_REFERENCE`	新建 Reference 表示并加入对应内部索引；Master 能力摘要变化时更新其向量
`PROMOTE`	为新 Skill 生成 Embedding，匹配或创建新簇；更新父 Master 与历史归属
`SPLIT`	根据重新分配的成员向量，重算受影响的簇中心与索引
`MERGE`	生成整合后能力表示，调整原有成员记录与簇中心
`ARCHIVE`	从活跃技能发现索引中移除，按成员关系更新受影响簇中心

实现中可以设置独立的 **Skill Index Manager**，统一执行 `re-embed → reassign → recompute → refresh`，避免各个变更操作维护不同步的索引。它只处理**受影响的 Skill、Reference 与簇**，通常不需要全池重新编码。

需要特别区分两种向量：

1. **历史成员向量：** 用于在线聚类中心和成员重新分配；
2. **当前 Master/Reference 向量：** 用于任务发现与簇内模块匹配。

主文件经 LLM 改写后，它的语义可能改变；因此不宜无条件用"最新 Master 的 Embedding"替代由历史已吸收能力构成的簇中心。当历史成员实际发生迁移时，再根据成员集合准确重算中心。

---

8. 端到端示例

以通用数据处理能力为例，初始技能池包含：

```
skills/
└── data-analysis/
    ├── SKILL.md
    └── references/
        ├── data-cleaning.md
        ├── visualization.md
        └── statistical-testing.md
```

阶段 A——控制独立 Skill 数量： 智能体连续获得若干关于实验统计的候选 Skill。在线聚类发现其与 `data-analysis` 能力簇相关，不立即创建多个独立 Skill，而是优先向 `statistical-testing.md` 追加差异化步骤。

阶段 B——控制 Master Skill 长度： 统计检验的假设选择、边界条件、结果解释等专项细节全部保存在 Reference 中。Master 只保留"何时加载统计检验模块"的索引，避免每次执行数据分析都加载全部统计方法。

阶段 C——触发晋升： 后续任务多次直接要求执行统计检验，而不依赖完整数据分析工作流。此时 `statistical-testing.md` 已具有独立输入、流程和输出，可以晋升。

```
skills/
├── data-analysis/
│   ├── SKILL.md
│   └── references/
│       ├── data-cleaning.md
│       └── visualization.md
└── statistical-testing/
    ├── SKILL.md
    └── references/
        └── test-selection.md
```

阶段 D——局部索引同步： 计算 `statistical-testing` 的新 Embedding，匹配其他现有簇或建立新簇；若父 Skill 的 `description` 缩小了适用范围，也同步重新编码。历史候选成员仅在发生真实迁移时从父簇移出。

这个例子说明：系统不是将 Skill 数量压缩到越少越好，而是根据**任务边界与能力复用关系**动态调整颗粒度。

---

9. 实现边界、风险与研究展望

9.1 当前版本明确不做的内容

本方案第一阶段**暂不加入自动化行为效果验证**，不要求每次 Reference 更新都重放任务，也不设计新旧 Skill 的 A/B 成功率门控。文中的"接纳"仅指管理流程完成了归属与内容整合，**不代表实际执行效果已被证明提升。**

为方便排查，仍建议保存修改前文件快照、变更原因、历史候选来源与版本号。这是基本的可追溯性管理，不等价于效果验证。

9.2 主要风险

风险	影响	初步缓解方向
语义相似但流程不兼容	强行融合导致职责混淆	相似度只负责召回，融合前分析操作关系
过度聚类	不同任务被塞入同一 Master	明确能力边界，定期触发拆分审查
Reference 爆炸	规模问题在内部重新出现	优先 `UPDATE`，定期 `MERGE` 与 `PROMOTE`
错误晋升或结构震荡	增加重复入口与路由不稳定	使用独立任务信号、结构冷却期与变更快照
语义中心漂移	新候选被归入错误簇	保存成员向量、去重计权、局部重算
错误经验扩散	整合后的操作策略可能无效	当前版本承认该限制；后续再研究行为验证

9.3 后续可以评估的研究问题（未实施）

为检验所提框架是否真正解决原始动机，未来可比较：

- **技能规模：** 活跃 Skill 数量、Reference 数量及其随任务量的增长曲线；
- **冗余程度：** 语义重复率、整合后的能力覆盖情况；
- **路由效率：** Skill 选择正确率、触发歧义与检索开销；
- **上下文成本：** 单任务加载的 Skill/Reference Token 数；
- **结构稳定性：** 晋升/拆分频次、重复合并率及版本震荡情况；
- **任务表现：** 在具备可复现实验条件后，再比较任务完成质量，避免只报告压缩率。

可设置三个递进基线：① 每次新增独立 Skill；② 在线聚类并持续扩展单个 Master；③ 在线聚类 + Master/References + 动态晋升/拆分。该设计有助于隔离每个机制对应的具体动机，但**当前未运行任何对比实验。**

---

10. 结论

本文围绕持续学习型智能体的 Skill 库规模控制问题，提出一个由动机逐步驱动的层次化自进化框架：首先，通过在线语义聚类将重复的候选 Skill 归入已有能力簇，缓解独立入口数量膨胀；随后，将聚类关系转化为“父 Skill 统一入口、子 Skill 承载专项能力”的层次组织；进一步利用 Master Skill 与 References 分离通用流程和操作细节，避免父 Skill 正文冗长；最后，通过基于任务独立性与簇内语义分化的晋升、拆分及合并机制，调整不再合理的父子关系，防止内部模块继续无序膨胀。局部 Embedding 重算与索引同步使上述结构调整能够反映到后续的技能发现和调用过程中。

本方案的核心思想是：Skill 自进化不仅意味着增加新技能或更新旧指令，更意味着根据长期积累的任务需求，持续调整能力的组织颗粒度。 理想的技能库既不应因经验积累而无限膨胀，也不应因过度压缩而失去必要的独立能力入口。

当前工作属于架构与方法提案，下一阶段重点在于实现在线聚类、Reference 整合和事件驱动的晋升/拆分流程；行为效果验证及性能结论留待后续研究。

参考资料

[1] Agent Skills. *Specification and Documentation*. https://agentskills.io/specification ; https://github.com/agentskills/agentskills

[2] EverMind AI. *Raven: The Harness of Harnesses for Composable Agentic Intelligence*. arXiv:2609.33439, 2026. https://arxiv.org/abs/2609.33439

> **引用说明：** [1] 支撑标准 Skill 文件结构与渐进式加载；[2] 用于说明经验驱动的 Skill 自进化研究背景。文中提出的"候选 Skill 在线聚类—Master/References 整合—动态晋升/拆分"是本文的组合设计方案，**并非直接复述上述资料的完整算法。**